

Enlear Academy · [Follow publication](#)

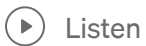
# Data Encryption on AWS

## Part 01 — Introduction

6 min read · Jan 11, 2020



Manoj Fernando



This blog post is related to [Data Encryption on AWS](#) youtube video.

Imagine that your server got hacked. Now, the hacker has full access to the sensitive data stored on the disk. You are in big trouble since you haven't encrypted that data and the hacker can do whatever he wants with your plain text data.

**Encryption is vital** if you deal with sensitive data that must not be accessed by unauthorized users. Regulations like GDPR (General Data Protection Regulation) instruct companies to encrypt both data at transit and data at rest. This article is about how to encrypt your data on AWS.

## Encryption at Rest vs in Transit

When you deliver your website over HTTPS by associating an SSL certification with your domain, the browser makes sure to encrypt the data in transit. The communication between the browser and the server is encrypted. However, as soon as the data (e.g. username and password) gets to the point where the SSL termination happens (At the server itself, Load Balancer, CloudFront, etc...) the encrypted data is decrypted. After that, the server is storing the plain text (e.g. username and password) in the server storage or in databases. If you want to avoid saving plain text, you have to enable encryption at rest.

## Encryption at Rest

This is about encrypting the data that you store in the backend servers and databases. There are two main methods to encrypt data at rest.

1. Client-Side Encryption
2. Server-Side Encryption

### Client-Side Encryption

As the name implies this method encrypts your data at the client-side before it reaches backend servers or services. You have to supply encryption keys  to encrypt the data from the client-side. You can either manage these encryption keys by yourself or use AWS KMS(Key Management Service) to manage the encryption keys under your control.

AWS provides multiple client-side SDKs to make this process easy for you. E.g. AWS Encryption SDK, S3 Encryption Client, DynamoDB Encryption Client etc...

### Server-Side Encryption

In Server-Side encryption, AWS encrypts the data on your behalf as soon as it is received by an AWS Service. Most of the AWS services support server-side encryption. E.g. S3, EBS, RDS, DynamoDB, Kinesis, etc...

All these services are integrated with AWS KMS in order to encrypt the data.

## AWS KMS

AWS KMS (Key Management Service) is the service that manages encryption keys on AWS. These encryption keys are called “**Customer Master Keys**” or CMKs for short. KMS uses Hardware Security Modules (Physical devices, commonly known as HSM) to store CMKs. AWS KMS is integrated with many AWS services and it uses AWS CloudTrail to track the usage logs of the keys for audit and compliance needs.

### Customer Master Keys(CMKs) VS Data Keys

CMKs are created and managed by AWS KMS. However, CMK is only used to encrypt a small amount of data less than 4KBs. AWS does not encrypt the gigabytes of data using CMK. If you have large data to encrypt, then use Data Keys.

**Data Keys** are generated from CMKs. There is a direct relationship between Data Key and a CMK. However, AWS does NOT store or manage Data Keys. Instead, you have to manage them.

---

Get Manoj Fernando's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

---

Look at the following diagram.

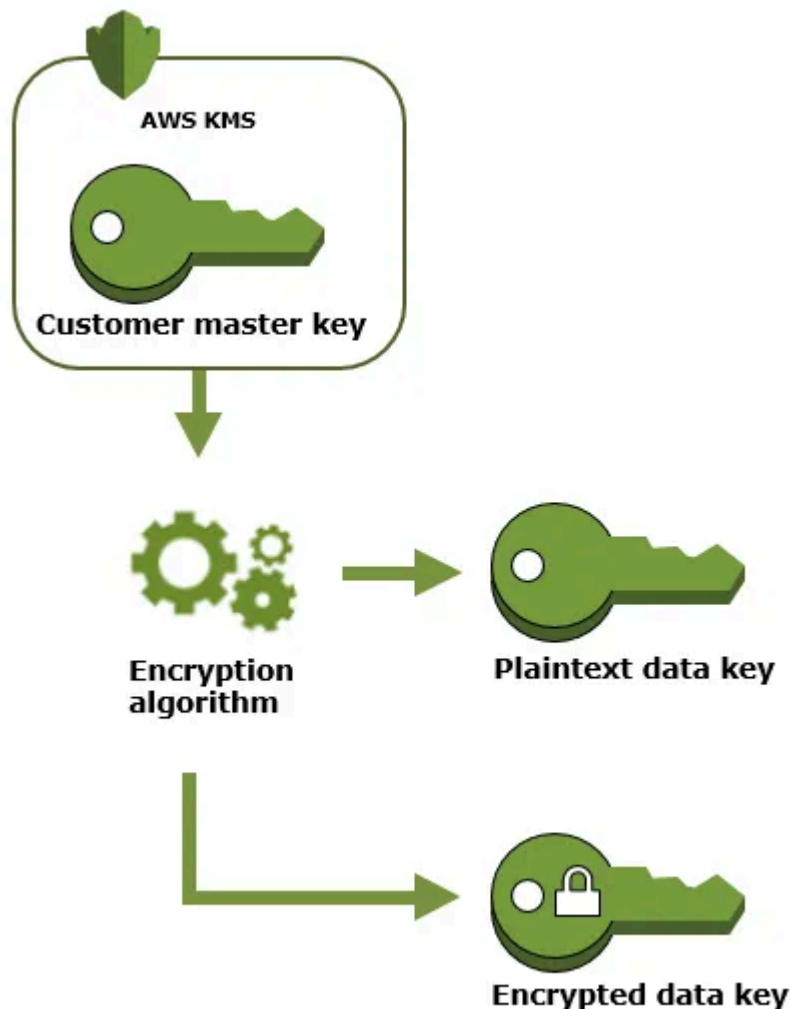


Image 1 — Generate Data Keys from a CMK (Ref — AWS Documentation)

You can use one Customer Master Key (CMK) to generate thousands of unique data keys. You can generate data keys from a CMK using two methods.

1. Generate both **Plaintext Data Key** and **Encrypted Data Key**
2. Generate **only the Encrypted Data Key**

Image-1 illustrates how to generate both plain-text and encrypted data keys using a CMK.

## Encrypt/Decrypt Data

Once you get the Plaintext data key and Encrypted data key from CMK, use the Plaintext data key to encrypt your data. After encryption, never keep the Plaintext data key together with encrypted data(Ciphertext) since anyone can decrypt the Ciphertext using the Plaintext key. So remove the Plaintext data key from the memory as soon as possible. You can keep the Encrypted data key with the Ciphertext. When you want to decrypt it, call the **KMS API** with the encrypted data

key and **KMS will send you the Plaintext key** if you are authorized to receive it. Afterward, you can decrypt the Ciphertext using the Plaintext key.

## Envelope Encryption

The method of encrypting the key using another key is called **Envelop Encryption**. By encrypting the key, that is used to encrypt data, you will protect both data and the key.

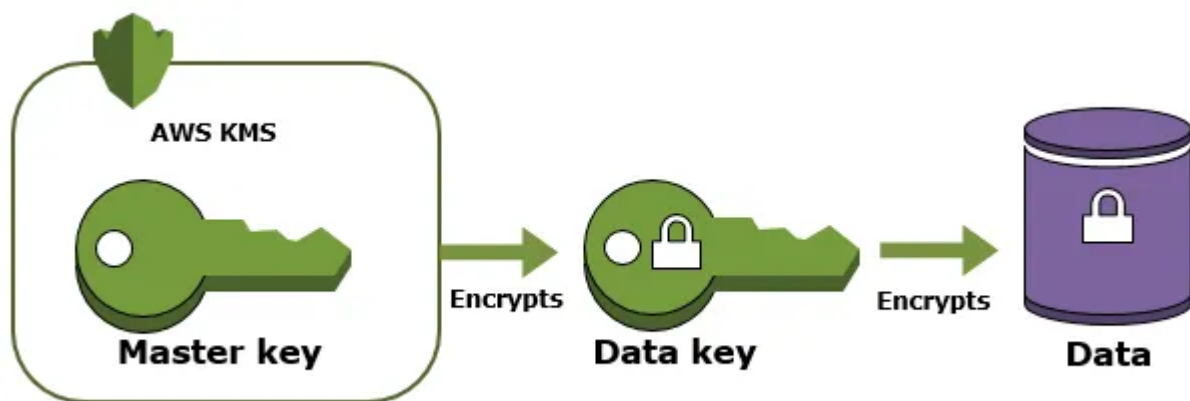


Image 2 — Envelop Encryption (Ref — AWS Documentation)

In AWS you can encrypt the Data key used to encrypt the Data with Customer Master Key(CMK). But, where do you store the CMK? AWS KMS will store it inside Hardware Security Module (HSM) with a greater level of protection. (HSM is compliant with FIPS 140–2 security standard)

## Key Policies

One of the powerful features in KMS is the ability to define permission separately for those **who use the keys and administrate the keys**. This is achieved using Key Policies.

```
{
  "Sid": "Enable IAM User Permissions",
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::111122223333:root"},
  "Action": "kms:*",
  "Resource": "*"
}
```

The above key policy is applied to the root user of the account. It allows full access to the CMK that this policy is attached. When it comes to other users and roles you

can manage key usage and key administration as follows.

```
{
  "Sid": "Allow use of the key",
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::111122223333:user/manoj"},
  "Action": [
    "kms:Decrypt",
    "kms:DescribeKey",
    "kms:Encrypt",
    "kms:GenerateDataKey*",
    "kms:ReEncrypt*"
  ],
  "Resource": "*"
}
```

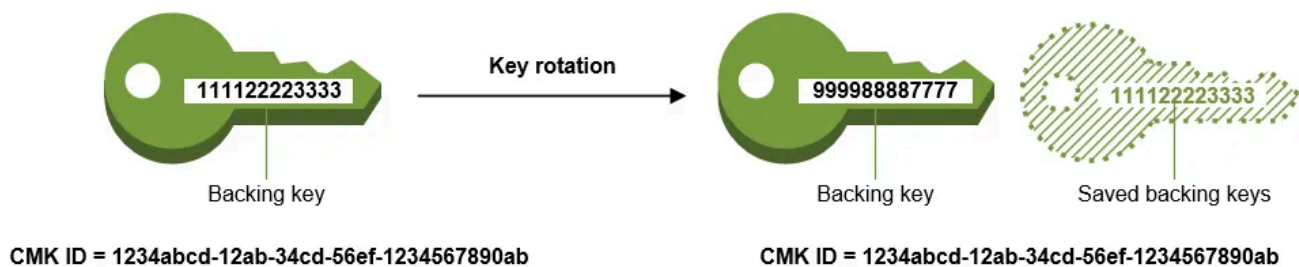
The above policy is applied to the IAM user ‘manoj’. Now he has permission to use the CMK for encryption and decryption. However, he is not allowed to administrate that CMK.

```
{
  "Sid": "Allow access for Key Administrators",
  "Effect": "Allow",
  "Principal": {"AWS": [
    "arn:aws:iam::111122223333:user/KMSAdminUser"
  ]},
  "Action": [
    "kms:Create*",
    "kms:Describe*",
    "kms:Enable*",
    "kms:List*",
    "kms:Put*",
    "kms:Update*",
    "kms:Revoke*",
    "kms:Disable*",
    "kms:Get*",
    "kms>Delete*",
    "kms:TagResource",
    "kms:UntagResource",
    "kms:ScheduleKeyDeletion",
    "kms:CancelKeyDeletion"
  ],
  "Resource": "*"
}
```

Now, the above key policy allows administrators to administrate the CMK that it is applied to. However, the administrator cannot use the key to Encrypt or Decrypt data.

## Key Rotation

Cryptographic best practices discourage extensive use of encryption keys. Because of that, AWS allows rotating the Customer Master Key(CMK). You can enable the **automatic renewal option** for the CMKs that you have created in KMS. The CMKs that you have created in KMS are commonly known as **Customer Managed CMKs**. Once you enable automatic key renewal, KMS renews the key's cryptographic material(Backing Key) in **every year**. However, CMKs managed by AWS are only **renewed every three years**. You cannot change the renewal frequency for AWS managed CMKs.



It is important to understand that AWS KMS saves references to the older backing keys when renewing. So that KMS is able to decrypt data or data keys that were generated by older versions of backing keys. Otherwise, those data/data keys can never be decrypted.

In the next post, let's discuss S3 and EBS encryption.

[Security](#)[AWS](#)[Encryption](#)[Follow](#)

**Published in Enlear Academy**

1.4K followers · Last published Nov 15, 2025

We provide high quality content on web development and cloud technologies for developers.



## Written by Manoj Fernando

837 followers · 118 following

AWS Community Hero | Cloud Architect | For more details find me on LinkedIn

<https://www.linkedin.com/in/manoj-fernando-b7351935>

---

### Responses (1)



---

See all responses